

基于 Termux 的多模型 QQ 群聊机器人系统

3099404236¹

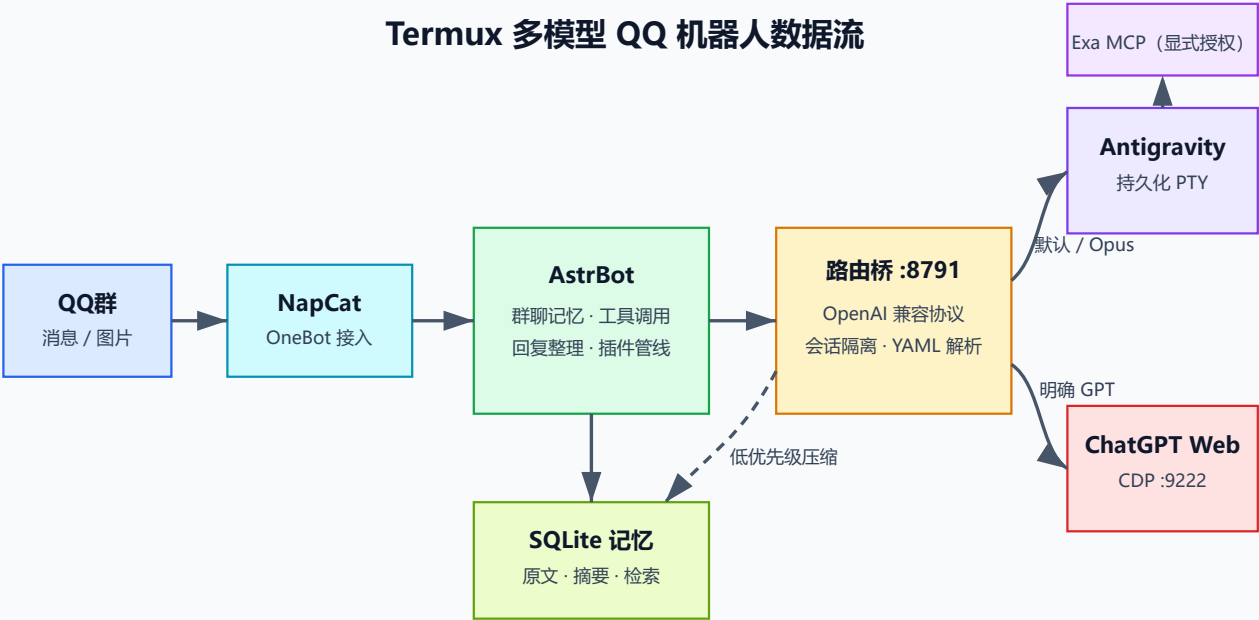
¹独立开发者

本报告介绍一套运行在 Redmi Android 手机上的完整 QQ 群聊 AI 机器人。系统以 NapCat 和 AstrBot 为入口与编排层，通过 OpenAI 兼容桥连接持久化 Antigravity 终端，并在用户明确指定路由到 Opus、Exa MCP 或 ChatGPT 网页会话。项目同时实现群消息持久化、异步结构化摘要、历史检索、长回复整理、图片回传、链路延迟观测和后台守护。公开代码来自 2026 年 8 月 1 日对生产手机的只读快照；账号、Cookie、OAuth 状态、API Key、聊天数据库、日志、二维码、浏览器 Profile 和模型会话记录均被排除。该实现证明了低成本 Android 设备可以承载多进程、可恢复的群聊 Agent 系统，也揭示了网页自动化、移动系统后台限制与第三方登录状态带来的工程脆弱性。

Termux | QQ bot | context engineering | multi-model routing | browser automation

 [GitHub: 3099404236/termux-multimodel-qq-bot](https://github.com/3099404236/termux-multimodel-qq-bot)

Termux 多模型 QQ 机器人数据流



所有服务默认仅监听 127.0.0.1；账号、Cookie、API Key 与聊天数据不进入公开仓库。

图 1 系统数据流。AstrBot 只面对一个 OpenAI 兼容入口，模型和会话路由由桥接层完成。

目标与约束

项目的目标不是在手机上直接运行大语言模型，而是把手机变成一台长期在线的轻量编排主机：QQ 消息进入后，由 AstrBot 管理事件、记忆和工具，再把真正的推理工作交给已经登录的外部模型界面。这样可以在没有小主机的情况下复用现有 Android 设备，同时保留群聊上下文、多轮对话和模型选择能力。

设计受到四个现实约束。第一，手机内存有限，不能为每条消息重新启动完整会话。第二，Antigravity 是全屏交互式终端，若停止读取输出会产生 PTY 背压。第三，QQ、Google 和 ChatGPT 都可能要求扫码或网页登录，凭据不能写入代码。第四，群聊消息并发、长回答和后台压缩不能互相阻塞。

系统架构

如图 1 所示，NapCat 负责 QQ 与 OneBot，AstrBot 负责群聊事件管线，路由桥负责协议转换和模型选择。默认请求进入 Antigravity 持久化 PTY；明确指定 GPT 的请求转到 ChatGPT 网页桥；明确授权 Exa 的请求由

Antigravity 在当前轮直接调用 Exa MCP。模型路由与对话标识分离，因此同一个群可以在不同模型间切换，而不会把几套上下文混在一起。

核心设计

持久化终端与背压控制.

Antigravity 的 TUI 会反复刷新整屏。若桥只在“等待回答”阶段读取 PTY，空闲刷新也可能塞满输出缓冲；下一次同步写入又可能塞满输入缓冲，形成双向死锁。当前实现为每个持久化会话配置专属工作线程，持续读取终端并维护滚动屏幕快照。其他请求只能进入该会话的队列，不能直接写终端。

写入、提交和进度分别有界：手机发布参数为 5 秒写入超时、20 秒提交检测、180 秒无进度超时。终端尺寸设为 160×50，减少每次全屏重绘的数据量；应用层保留 256 屏快照，不依赖终端滚动区保存历史。异常或取消时会话被关闭并等待线程退出，重试使用新会话，避免孤儿线程继续占用同一 PTY。

群聊记忆与自主检索.

组件	默认端口	职责
NapCat	6099	QQ 登录、消息接入与 OneBot 转发
AstrBot	6185	事件管线、工具、记忆、插件和回复发送
Antigravity 路由桥	8791	OpenAI 协议、模型路由、PTY 会话和输出解析
ChatGPT 网页桥	8766	CDP 页面控制、多轮复用、文本与图片返回
Chrome CDP	9222	控制已经登录的 ChatGPT 标签页

表 1 本机服务与职责。所有端口默认只监听回环地址。

每条群消息写入本地 SQLite。默认向模型注入最近 18 条原文和最近 3 批摘要；每 30 条消息形成一个待压缩批次，群聊安静 90 秒后异步启动结构化摘要。摘要固定包含主题、事实与决定、参与者立场、未解决问题、资源和检索关键词，空字段必须显式为空，从而降低自由摘要静默丢失关键信息的概率。

历史并不只靠固定摘要注入。AstrBot 还向模型提供搜索群历史的函数；模型在遇到“之前说过什么”、人物观点或缺少背景时可以检索原文与摘要。压缩会话使用低优先级标识，路由器禁止它抢占 Opus 或 GPT 通道。

显式模型与搜索授权。

路由器只读取最新用户消息的明确意图。用户明确要求 Opus 时进入独立 Opus 会话；明确要求 GPT 时进入 ChatGPT 网页桥；否则使用默认 Antigravity 模型。类似地，普通“搜索一下”只允许内置网页搜索，只有明确点名 Exa 才允许调用 Exa MCP。否定表达会覆盖关键词匹配，防止“不要用 GPT”或“不要用 Exa”被误判。

模型输出被约束为一个严格 YAML 信封：要么是用户可见的最终文本，要么是一次上游工具调用。桥只解析信封内部内容，过滤思考标题、搜索状态、token 统计和终端装饰，再交回 AstrBot。

ChatGPT 网页会话。

ChatGPT 网页桥直接使用 Chrome DevTools Protocol 操作已经登录的页面。为避免长文本逐字符输入导致 ProseMirror 反复重排，桥把文本按小块生成粘贴事件；同时把单块控制在网页“转为附件”的阈值以下。它根据停止按钮、回答增长和稳定窗口判断生成进度，而不是设置过短的总超时。

网页会话默认最多复用 10 轮或 180000 字符，超过阈值后新建对话，防止 DOM 与上下文无限增长。错误卡片、残留草稿、发送失败和图片型回答都有单独检测。图片被下载到 proot 可见的临时目录，再转换成 AstrBot 可以发送的路径。

手机部署与运行

系统由 Termux 外层和 proot Ubuntu 内层共同组成。NapCat、Chrome 和 ChatGPT 网页桥运行在 Termux 侧；AstrBot 与 Antigravity 路由桥运行在 Ubuntu 侧。启动脚本通过 proot 将 /dev、/proc 绑定进 Ubuntu，并

用 termux-wake-lock 降低休眠影响。NapCat 登录状态由独立 watchdog 判断，自动重启默认关闭，避免安静群聊被误判为离线。

可靠性与隐私边界

长回复在 QQ 发送前按 UTF-8 字节拆分，保证字符不会被切坏，并把每个合并转发节点控制在安全大小。延迟插件记录消息接收、模型调用、桥接、工具和发送阶段时间；栈监控器检查 6185、8791、8766 和 9222 的健康状态。若所有重试失败，AstrBot 会返回可见错误，而不是让用户无限等待。

公开仓库不包含任何运行数据。导出时排除了 AstrBot data/、日志、数据库、PID、Antigravity 工作目录与会话、桥依赖目录、ChatGPT 浏览器 Profile 和 Cookie。随后又扫描了密钥模式和设备标识；两个脚本中的 QQ 号默认值被替换为必须在手机本地提供的 NAPCAT_UIN 环境变量。

局限性

第一，ChatGPT 网页桥依赖页面 DOM 和选择器，网页更新后可能失效，也存在服务条款与账号风险；正式系统应优先使用官方 API。第二，Android 厂商的后台管理、低内存回收和网络代理仍可能终止进程，wake lock 与最近任务锁定只能降低概率。第三，模型路由目前主要依赖用户显式指令，尚未引入独立分类模型。第四，结构化摘要仍会损失细节，需要用真实失败案例持续调整字段和检索策略。第五，本报告记录的是一个可运行工程快照，不是对吞吐量或回答质量的受控基准测试。

复现与验证

公开仓库提供完整脱敏源码、环境变量示例、启动与守护脚本、关键单元测试以及本报告源文件。复现者需要自行安装并登录 NapCat、Antigravity 与 ChatGPT，不能从仓库恢复作者账号状态。核心 Python 检查使用 ruff 和 pytest；网页桥可用 node --check 验证语法；Typst 报告由本地脚本和 GitHub Actions 构建。

项目代码：github.com/3099404236/termux-multimodel-qq-bot。

层	运行环境	公开仓库中的实现
QQ 接入	Termux/proot	deployment/termux/ 与 NapCat 外部安装
机器人核心	Ubuntu	仓库根目录、astrbot/、plugins/
默认模型桥	Ubuntu	bridges/antigravity/
GPT 网页桥	Termux	bridges/chatgpt-web/
报告与验证	电脑/CI	paper/、tests/、GitHub Actions

表 2 运行层与源码对应关系。